

OSCIRIS Protocol

Orchestrated Secure Compute Infrastructure for Resilient Intelligent Sovereignty (OSCIRIS) Protocol

OSCIRIS Labs

Whitepaper v0.1
May 2026

Abstract

OSCIRIS Protocol is a decentralized secure compute protocol for privacy-preserving AI training, inference, RAG, evaluation, and fine-tuning. The protocol is designed for enterprises and sovereign institutions that need access to distributed GPU and compute liquidity without surrendering control of sensitive datasets, model artifacts, jurisdictional requirements, or audit obligations. OSCIRIS combines off-chain AI execution with on-chain staking, escrow, verification records, challenge windows, slashing, and insurance-pool routing. Its privacy layer, Data Shadow Protocol (DSP), transforms sensitive data or model features into task-preserving shadow representations and validates them against reconstruction, membership inference, property inference, and manifest-integrity checks. The initial economic design uses ZEN as provider collateral and USDC or other stable-value rails for customer payments and provider rewards.

Table of Contents

- [Executive Thesis](#)
- [Problem and Market Context](#)
- [Related Work and Competitive Landscape](#)
- [Solution Overview](#)
- [Target Users and Workloads](#)
- [Formal System Model](#)
 - [Actors](#)
 - [Security and Proof Boundaries](#)
- [Design Principles](#)
- [System Architecture](#)
 - [Control Plane and Compute Plane](#)
- [Protocol Mechanics](#)
 - [End-to-End Job State Machine](#)
 - [Scheduler Objective](#)
 - [Reputation Update](#)
- [Threat Model](#)
- [Privacy and Security Model](#)
 - [Post-Quantum Security Boundary](#)
- [Data Shadow Protocol](#)
 - [DSP Encoder](#)
 - [Guarded Feature Suppression](#)
 - [Utility Objective](#)
 - [Empirical Leakage Metrics](#)
 - [Random Projection Utility Intuition](#)
 - [Formal DP Shadow Release](#)

- [Formal DP Model Release](#)
 - [Result-Backed Math Check](#)
 - [DSP Claim Boundary](#)
- [Verification and Trust Model](#)
 - [Verifier Acceptance Rule](#)
 - [Redundancy and Spot-Check Detection](#)
 - [False Acceptance Under Verifier Error](#)
 - [Challenge Game](#)
 - [Slashing and Insurance Routing](#)
- [Blockchain Architecture and Horizen Implementation](#)
 - [Contract Invariants](#)
- [Token Economics and Incentive Design](#)
 - [Stake Tiers](#)
 - [Reward Formula](#)
 - [Reward Allocation](#)
 - [Effective Stake Cap](#)
 - [Sustainable Pricing Condition](#)
 - [Provider Participation Condition](#)
 - [Why No Native Token Initially](#)
- [Governance, Compliance, and Enterprise Controls](#)
- [Validation, Proof Status, and Claim Ledger](#)
 - [Claim Ledger](#)
- [Roadmap](#)
- [Risks and Limitations](#)
- [Conclusion](#)

Executive Thesis

AI infrastructure is becoming a compute, energy, sovereignty, and trust problem. The demand for AI training and inference is growing faster than the supply of reliable, affordable, and jurisdictionally appropriate compute. Centralized hyperscale clouds provide mature infrastructure, but they also concentrate control over data, pricing, energy demand, and model deployment. At the same time, a large global pool of underutilized GPUs and compute nodes exists across creator workstations, gaming rigs, universities, small data centers, enterprise clusters, edge sites, and former mining facilities.

The opportunity is not merely to create another cheap GPU marketplace. The defensible opportunity is to create a privacy-preserving, energy-aware, and auditable AI compute network that can serve enterprise and sovereign workloads. OSCIRIS begins with practical workloads: embeddings, RAG indexing, batch inference, model evaluation, synthetic-data generation, LoRA and QLoRA fine-tuning, and domain-specific small-model adaptation. Full frontier-model pretraining is explicitly out of scope for the initial protocol.

Blockchain is not required to run AI workloads. It is required for the full open-network economics: programmable collateral, job escrow, proof records, challenge windows, verifier incentives, slashing, insurance pools, and portable reputation across independent providers. In OSCIRIS, AI computation remains off-chain. The chain coordinates trust and settlement.

Problem and Market Context

Enterprise AI buyers need more than raw GPU access. They require data controls, auditability, privacy modes, compliance packaging, predictable billing, verified outputs, provider accountability, and jurisdiction-aware execution. Distributed GPU networks can increase supply, but they introduce trust risks: fake hardware, unreliable nodes, fabricated outputs, malicious updates, colluding providers, data leakage, storage non-retrievability, and poor enterprise procurement fit.

The core research and product problem is:

How can distributed AI compute become trustworthy enough for enterprise training and inference while remaining cost-effective, privacy-preserving, and viable in energy-constrained markets?

OSCIRIS addresses this problem through a layered architecture that separates:

- private AI execution from public settlement,
- privacy classification from raw GPU pricing,
- verifier-backed proof records from subjective output claims,
- stable customer payments from volatile provider collateral,
- permissioned enterprise pools from open public workloads.

Related Work and Competitive Landscape

OSCIRIS sits at the intersection of decentralized compute, privacy-preserving machine learning, GPU cloud infrastructure, verifiable execution, and blockchain settlement. The relevant landscape includes decentralized GPU marketplaces, specialized AI subnetworks, storage networks, TEE-based confidential computing, federated-learning systems, and traditional cloud GPU providers.

Category	Representative Systems	OSCIRIS Differentiation
Decentralized GPU marketplaces	Akash, io.net, Aethir	OSCIRIS adds privacy-tier routing, DSP leakage gates, verifier-backed receipts, enterprise controls, and ZEN-backed slashing rather than only raw compute matching.
AI incentive networks	Gensyn, Bittensor	OSCIRIS focuses on enterprise and sovereign workloads with stable payments, audit trails, permissioned pools, and explicit data-governance controls.
Storage and data networks	IPFS, Filecoin	OSCIRIS uses storage commitments and artifact manifests as part of a compute workflow; the primary protocol object is verified AI work, not storage alone.
Confidential-compute systems	GPU TEEs, CPU TEEs, secure enclaves	OSCIRIS integrates confidential execution where available while keeping DSP, verifier quorums, and challenge economics as independent controls.
Federated-learning systems	Institutional FL deployments	OSCIRIS treats federated execution as one privacy tier inside a broader secure compute network with settlement, incentives, auditability, and provider accountability.

The resulting position is not a generalized cloud replacement. OSCIRIS is a secure AI compute protocol for workloads where privacy, verification, jurisdiction, and economic accountability are part of the product.

Solution Overview

OSCIRIS Protocol coordinates distributed AI compute through three connected systems:

- an **off-chain compute network** that runs AI workloads on provider GPUs, vetted clusters, confidential nodes, or institutional compute;
- a **privacy and verification layer** that applies DSP, attestation, checkpointing, evaluator nodes, challenge windows, and audit records;
- an **on-chain settlement layer** that handles provider staking, stablecoin job escrow, verifier records, slashing, insurance routing, and reputation anchors.

The product surface is a managed secure AI workspace. A client submits a job, selects a privacy tier, funds escrow, receives verified outputs, and exports audit evidence. The decentralized network is visible where transparency and accountability matter, but enterprise users interact primarily with datasets, models, policies, metrics, and billing.

OSCIRIS is not designed to run AI computation inside smart contracts. Smart contracts enforce economic and state-transition rules. AI execution remains off-chain, where GPUs, storage systems, runtimes, and privacy controls can operate efficiently.

Target Users and Workloads

The recommended market posture is Africa-first in demand and global in compute supply. African enterprises and public institutions face acute constraints around power reliability, cloud concentration, data sovereignty, cost, and regulatory trust. Global GPU liquidity is necessary because local GPU density is not yet sufficient to satisfy broad demand.

The first buyers are institutions that need private AI workflows but cannot accept opaque, unauditable infrastructure. Initial ideal customer profiles are:

- banks and fintechs needing private RAG, compliance assistants, and domain adaptation;
- telcos needing call-center automation, local-language intelligence, and privacy-aware analytics;
- universities and research institutions needing affordable model evaluation and fine-tuning;
- public-sector teams needing local data control and auditability;
- energy and infrastructure partners exploring AI compute zones.

OSCIRIS does not begin with frontier-scale pretraining. The first network focuses on workloads that tolerate distributed execution and can be verified economically.

Workload	Initial Fit	Rationale
Embeddings and RAG indexing	Very high	Batchable, commercially useful, straightforward to verify through deterministic inputs and retrieval metrics.
Batch inference	Very high	Can be routed, replicated, and spot-checked.
Model evaluation and red teaming	High	Naturally parallel and easy to split across evaluator nodes.
Synthetic-data generation	High	Batchable and tolerant of asynchronous execution, with policy and quality review.
LoRA and QLoRA fine-tuning	High	Practical on scoped models and vetted GPU nodes with checkpoint-level verification.
Federated learning	Medium-high	Strong privacy fit for institutions, but requires robust aggregation and poisoning controls.
Full LLM pretraining	Low initially	Requires dense interconnects, synchronized clusters, high uptime, and expensive verification.

Formal System Model

OSCIRIS is modeled as a hybrid system with an on-chain coordination plane and an off-chain compute plane. The protocol does not assume that the blockchain can verify arbitrary AI execution directly. Instead, it assumes the chain can enforce collateral, escrow, signatures, challenge periods, and state transitions whose inputs are hashes, attestations, verifier results, and dispute outcomes.

Actors

Let:

- $\mathcal{C}$ be the set of clients creating AI jobs;
- $\mathcal{P} = \{P_1, \dots, P_m\}$ be compute providers;
- $\mathcal{V} = \{V_1, \dots, V_\ell\}$ be verifier or evaluator nodes;
- $\mathcal{K}$ be the smart-contract suite deployed on Horizen;
- $\mathcal{A}$ be the artifact space containing datasets, encrypted chunks, model checkpoints, runtime logs, output files, and evaluation reports;
- $\mathcal{G}$ be the governance and enterprise-control layer, including allowlists, privacy tiers, legal terms, incident rules, and data retention policies.

Provider P_i has state:

$$p_i = (s_i, c_i, r_i, u_i, q_i, z_i, \Pi_i),$$

where s_i is effective ZEN stake, c_i is compute capacity, r_i is non-transferable reputation, u_i is recent uptime, q_i is observed quality, z_i is jurisdiction and compliance metadata, and Π_i is the set of supported privacy tiers. Stake is necessary for participation, but reputation and verified performance determine long-run job priority.

Definition 1 (Artifact manifest). An artifact $A \in \mathcal{A}$ is represented on-chain only by a manifest:

$$M_A = (h_A, h_{\text{chunks}}, h_{\text{policy}}, h_{\text{runtime}}, t_{\text{created}}, \sigma_{\text{origin}}),$$

where $h_A = \text{Hash}(A)$, h_{chunks} is a Merkle root over encrypted or content-addressed chunks, h_{policy} commits to data-handling rules, h_{runtime} commits to the runtime image or measurement, and σ_{origin} is the origin signature. The chain stores commitments and signatures, not the raw artifact.

Definition 2 (OSCIRIS job). An OSCIRIS job is a tuple:

$$J = (j, c, h_D, h_{\theta_0}, \tau, \pi, b, T, Q, \Lambda, \Gamma),$$

where j is a job identifier, $c \in \mathcal{C}$ is the client, h_D commits to the dataset or data manifest, h_{θ_0} commits to the base model or initial weights, τ is the workload type, π is the privacy policy, b is the stablecoin budget, T is the deadline, Q defines acceptance metrics, Λ defines slashing conditions, and Γ defines the challenge window and verifier quorum.

The job output is accepted only if the protocol records a receipt:

$$\rho_J = (j, h_{\text{out}}, h_{\text{log}}, h_{\text{eval}}, \Sigma_V, t_{\text{final}}, \text{status}),$$

where h_{out} commits to the output artifact, h_{log} commits to execution logs, h_{eval} commits to verification results, and Σ_V is a set of verifier signatures satisfying the quorum rule for the job tier.

Security and Proof Boundaries

The whitepaper uses three levels of proof language:

- **Formal proof:** a mathematical statement that follows under explicit assumptions, such as a differential-privacy bound for a calibrated Gaussian mechanism.
- **Protocol proof sketch:** an argument that the on-chain state machine enforces a property if its inputs, signatures, and verifier outcomes are valid.
- **Empirical evidence:** benchmark results from the local OSCIRIS validation lab. These are evidence for product direction, not universal production guarantees.

This distinction matters. DSP can make formal differential-privacy claims only for mechanisms whose randomness, clipping, sensitivity, and composition match the proof assumptions. Other DSP transforms can still be valuable engineering tools, but their privacy posture is empirical and adversarially tested rather than formally proven.

Design Principles

1. **AI computation stays off-chain.** Smart contracts coordinate commitments, staking, escrow, verification records, disputes, and slashing. They do not execute model training or inspect raw private data.
2. **Privacy is tiered.** Public workloads can use broad compute liquidity. Sensitive workloads require vetted providers, trusted execution, federated learning, secure aggregation, or local-only execution.
3. **ZEN is a bond, not a forced payment rail.** Providers stake ZEN to qualify for work and lose ZEN for objective failures. Customers pay in USDC or stable-value rails so the procurement experience feels closer to cloud infrastructure than to a speculative token product.
4. **Reputation must be earned.** Stake unlocks participation, but it does not fully determine job priority. Provider performance, uptime, verification outcomes, privacy-tier history, and dispute behavior drive reputation.
5. **Enterprise trust requires governance.** Token incentives are not enough for banks, telcos, hospitals, governments, or regulated enterprises. OSCIRIS supports provider onboarding, audit exports, incident handling, acceptable-use policy, data-retention controls, and permissioned pools.

System Architecture

OSCIRIS Protocol is organized as a layered secure compute network.

Layer	Function
Client and enterprise layer	Workspaces, dataset registration, job creation, model selection, privacy policy, billing, and audit access.
Data governance layer	Dataset manifests, consent metadata, jurisdiction labels, retention rules, and access policies.
Encrypted storage layer	Chunking, encryption, erasure coding, content addressing, Merkle roots, and retrieval or storage proofs.
Compute provider layer	Retail GPUs, prosumer rigs, universities, small data centers, enterprise clusters, and trusted compute nodes.
Privacy compute layer	Public compute, vetted nodes, TEE or confidential compute, federated learning, secure aggregation, and local-only workflows.
Verification layer	Node attestation, benchmark challenges, checkpoint hashing, output validation, evaluator nodes, and verifier signatures.
Orchestration layer	Job scheduling, batching, model placement, GPU matching, checkpointing, failover, energy-aware routing, and privacy-tier routing.
Consensus and settlement layer	ZEN staking, USDC escrow, proof records, challenge windows, slashing, insurance routing, and reputation anchors.
Developer API layer	OpenAI-compatible inference endpoints, SDKs, model registry, dashboard, audit exports, and enterprise integration tools.

Control Plane and Compute Plane

OSCIRIS separates the protocol into two mathematical planes.

- The **control plane** is the smart-contract state machine: provider registration, staking, escrow, assignment commitments, verifier receipts, challenge windows, slashing, insurance-pool routing, and reputation anchors.
- The **compute plane** is the off-chain execution network: data preparation, DSP transformation, model execution, checkpointing, verifier evaluation, and artifact publication.

At time t , the control-plane state is:

$$S_t = (\mathcal{P}_t, \mathcal{J}_t, \mathcal{E}_t, \mathcal{R}_t, \mathcal{I}_t),$$

where $\mathcal{P}_t$ is provider state, $\mathcal{J}_t$ is active job state, $\mathcal{E}_t$ is escrow state, $\mathcal{R}_t$ is reputation state, and $\mathcal{I}_t$ is insurance-pool state. A valid on-chain action a_t induces:

$$S_{t+1} = \Gamma_{\mathcal{K}}(S_t, a_t),$$

where $\Gamma_{\mathcal{K}}$ is the transition function encoded by the contract suite. Valid actions include provider registration, stake deposit, job creation, assignment acceptance, receipt submission, challenge submission, challenge resolution, payout, slashing, and withdrawal.

Proposition 1 (On-chain minimality). *If raw datasets, model weights, intermediate tensors, and private logs are represented on-chain only by hashes, Merkle roots, signatures, and policy commitments, then the chain can*

coordinate economic accountability without becoming a data processor for private AI workloads.

Proof. For each artifact A , the chain receives M_A , not A . The validation predicate checks signature validity, hash equality, quorum status, challenge expiry, and stake/escrow balances. None of those checks requires raw data or model tensors. The chain therefore enforces accountability over commitments while artifact custody and processing remain off-chain. $\square$

Protocol Mechanics

End-to-End Job State Machine

A job proceeds through the following state machine:

Draft $\rightarrow$ Funded $\rightarrow$ Matched $\rightarrow$ Committed
 $\rightarrow$ Executing $\rightarrow$ Submitted $\rightarrow$ Verifying
 $\rightarrow$ Challengeable $\rightarrow$ {Settled, Slashed, Cancelled}.

The settlement rule for job J is:

$$\text{settle}(J) = \begin{cases} \text{pay}(P_i, b_J) & \text{if } \text{valid}(\rho_J) = 1 \text{ and } \text{challenge}(J) = 0, \\ \text{slash}(P_i, \lambda_J) & \text{if } \text{violation}(J) = 1, \\ \text{refund}(c, b_J) & \text{if } \text{expired}(J) = 1 \text{ without valid receipt.} \end{cases}$$

This makes the protocol economically deterministic even though the underlying AI computation is probabilistic and off-chain.

Scheduler Objective

For each job J , the orchestrator chooses an assignment matrix $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ means provider P_i is assigned to job shard or task j . The routing objective is:

$$\begin{aligned} \min_x \quad & \sum_{i,j} x_{ij} (\alpha C_{ij} + \beta L_{ij} + \gamma E_{ij} + \eta RISK_{ij} - \mu REP_i) \\ \text{s.t.} \quad & \sum_i x_{ij} \geq k_j & \forall j, \\ & \sum_j x_{ij} g_j \leq c_i & \forall i, \\ & s_i \geq s_{\min}(\pi_j) & \forall i, j \text{ with } x_{ij} = 1, \\ & \pi_j \in \Pi_i & \forall i, j \text{ with } x_{ij} = 1, \\ & z_i \in Z_j & \forall i, j \text{ with } x_{ij} = 1. \end{aligned}$$

Here C_{ij} is expected cost, L_{ij} is latency, E_{ij} is energy or carbon penalty, $RISK_{ij}$ is privacy/compliance risk, REP_i is a reputation score, g_j is required compute, c_i is capacity, $s_{\min}(\pi_j)$ is the minimum stake for privacy tier π_j , and Z_j is the allowed jurisdiction set. This is the mathematical basis for the whitepaper claim that OSCIRIS is not merely a GPU marketplace: routing depends on privacy, jurisdiction, energy, reliability, and verification risk.

Reputation Update

Reputation is non-transferable and slowly moving:

$$r_i(t+1) = (1 - \omega)r_i(t) + \omega(aq_i(t) + bu_i(t) + cv_i(t) - d\chi_i(t)),$$

where q_i is quality, u_i is uptime, v_i is verifier agreement, and χ_i is a violation indicator. Stake s_i gates access, but r_i determines long-run assignment preference. This prevents a provider from buying high trust instantly by acquiring more ZEN.

Threat Model

OSCIRIS assumes an open or semi-open provider network where participants can be rational, faulty, or adversarial. The protocol does not assume that stake, identity, or cloud-style terms alone make compute trustworthy. The threat model therefore covers infrastructure, privacy, verification, economic, and governance risks.

Threat	Failure Mode	Primary Controls
Malicious provider	Fabricated outputs, skipped computation, poisoned updates, hidden model changes	Verifier quorums, checkpoint hashes, canary tasks, slashing, reputation loss.
Unreliable provider	Timeouts, partial execution, poor uptime, lost checkpoints	Stake tiers, retry routing, uptime scoring, escrow release conditions.
Data exfiltration	Raw data leakage, artifact theft, unauthorized retention	DSP, encryption, policy manifests, vetted pools, TEE modes, audit records.
Colluding participants	Providers or verifiers coordinating to pass invalid work	Independent verifier operators, weighted quorum rules, challenge windows, enterprise allowlists.
Sybil participation	Many low-quality nodes attempting to gain assignments	ZEN collateral, non-transferable reputation, hardware checks, tiered access.
Bad attestation	False runtime claims or stale measurement reports	Attestation registry, approved runtime hashes, expiry rules, verifier review.
Economic manipulation	Underpriced jobs, slash evasion, stake concentration	Escrow pricing constraints, effective stake caps, insurance thresholds, challenge bonds.
Bridge or stablecoin risk	Settlement disruption, liquidity failure, contract address changes	Deployment-time address verification, stable-value rail abstraction, treasury and legal review.
Governance capture	Unsafe parameter changes or privacy-tier dilution	Enterprise controls, staged governance, audit requirements, conservative upgrade policy.

Out of scope threats are stated explicitly. OSCIRIS does not claim to prevent all hardware side channels, compromised client endpoints, malicious customer data, legal coercion, zero-day vulnerabilities in GPU drivers, or failures in external bridges and stablecoin issuers. Those risks are handled through deployment policy, provider selection, legal controls, insurance limits, and ongoing security review.

Privacy and Security Model

OSCIRIS security is layered rather than dependent on one mechanism. DSP reduces direct data exposure before selected distributed workloads. Cryptography protects transport, keys, signatures, manifests, and stored artifacts. Verification detects invalid or low-quality work. Staking and slashing make objective failures economically costly. Enterprise controls restrict the provider set and operational policy for regulated workloads.

The privacy stack has four boundaries:

- **Representation boundary:** DSP creates task-preserving shadow representations and model-release modes with explicit leakage gates.
- **Execution boundary:** privacy tiers route work to public nodes, vetted providers, confidential runtimes, federated workflows, or local-only execution.
- **Verification boundary:** verifiers receive only the artifacts, evaluation sets, manifests, scores, and logs required for their role.
- **Settlement boundary:** smart contracts store commitments, signatures, scores, challenge states, and balances, not private datasets or model internals.

Post-Quantum Security Boundary

DSP is a privacy and representation-control layer, not a post-quantum cryptographic primitive. Its main tools are guarded transforms, random projection, differential privacy, empirical attack testing, and release-policy controls. OSCIRIS treats DSP as post-quantum compatible because it can operate alongside post-quantum or hybrid post-quantum cryptography for artifact encryption, transport security, key establishment, signatures, and long-term audit integrity.

NIST's first finalized post-quantum cryptography standards are FIPS 203, FIPS 204, and FIPS 205 . FIPS 203 specifies ML-KEM for key encapsulation . FIPS 204 specifies ML-DSA for digital signatures . FIPS 205 specifies SLH-DSA, a stateless hash-based signature standard .

OSCIRIS separates the cryptographic stack into four layers:

- **DSP privacy layer:** shadow transforms, DP model release, empirical leakage gates, and privacy-tier routing;
- **PQC transport and key establishment:** hybrid TLS or application key exchange using standardized or deployment-ready PQC KEMs;
- **PQC signatures:** artifact manifests, verifier receipts, provider attestations, and governance records signed with PQC or hybrid signature schemes;
- **Crypto-agility:** manifests record algorithm identifiers, parameter sets, key versions, and migration status so the protocol can rotate cryptographic primitives without changing the DSP math.

In this architecture, DSP reduces direct data exposure and supports scoped formal DP claims; PQC protects cryptographic channels and signatures against future quantum-capable adversaries.

Data Shadow Protocol

Data Shadow Protocol is the OSCIRIS privacy layer. DSP transforms sensitive data or model features into task-preserving shadow representations before distributed training or evaluation. The design goal is to retain enough utility for approved AI tasks while reducing direct exposure of raw features.

The current prototype validates DSP through:

- utility retention in shadow-space learning,
- known-pair linear and nonlinear reconstruction attacks,
- confidence-gap membership inference,
- property inference from shadow records,
- tensor-shadow compression experiments,
- colluding shard leakage curves,
- Merkle manifest verification,
- scoped differential privacy model-release experiments.

DSP is defined precisely. Most shadow transforms are empirical privacy mechanisms, not formal privacy proofs. The current strongest formal local result is a scoped differential privacy model-release path:

Configured feature guarding, per-example gradient clipping, fresh Gaussian noise on averaged full-batch gradients, basic composition across epochs, and final model release as post-processing of the composed noisy gradients.

The confirmed local DP model-release result uses epsilon 25, delta 10^{-5} , clip norm 1.0, 80 epochs, configured guard mode, and 15 seeds. It reaches a selected-release pass rate of 12 out of 15 with mean AUC retention of 0.905. This is evidence for a scoped model-release claim, not a full enterprise privacy proof. It does not cover key compromise, worker memory inspection, side channels, malicious updates, encrypted transport, or production governance.

DSP Encoder

Let $X \in \mathbb{R}^{n \times d}$ be a tabular or embedding dataset with n records and d features. A DSP encoder is a keyed randomized map:

$$C_K: \mathbb{R}^{n \times d} \rightarrow \mathbb{R}^{n \times m},$$

where K is an origin-controlled key and m is the shadow dimension. The general encoder family is:

$$Y = C_K(X) = \psi \left(G_\phi(X) A_K + \mathbf{1} b_K^\top + \Xi \right).$$

Here:

- G_ϕ is an optional guard that suppresses sensitive or correlated features according to dataset metadata ϕ ;

- $A_K \in \mathbb{R}^{d \times m}$ is a keyed projection matrix, such as sparse sign, Gaussian, orthogonal, feature-hash, or low-rank projection;
- $b_K \in \mathbb{R}^m$ is an optional keyed bias;
- Ξ is noise, deterministic for benchmark replay or fresh for formal DP modes;
- ψ is an optional post-processing function, such as identity, tanh, sign binarization, or tensor-network compression.

The practical DSP claim is not that every C_K is formally private. The claim is narrower: DSP gives OSCIRIS a measurable privacy-control surface. Each candidate C_K is evaluated against explicit utility and leakage metrics before it is eligible for a privacy tier.

Guarded Feature Suppression

Let $S \subseteq \{1, \dots, d\}$ be the configured sensitive-feature set and let $D_S \in \mathbb{R}^{d \times d}$ be a diagonal mask with:

$$(D_S)_{kk} = \begin{cases} 0 & k \in S, \\ 1 & k \notin S. \end{cases}$$

The deterministic guard is:

$$G_\phi(X) = XD_S.$$

Correlation-guarded variants expand S using an origin-node fit:

$$S^+ = S \cup \{k: |\text{corr}(X_k, X_s)| \geq \rho \text{ for some } s \in S\},$$

subject to a configured maximum number of proxy features. This is not a standalone privacy theorem. It is an auditable mitigation: direct sensitive columns and high-correlation proxies are removed before projection, and the resulting utility/privacy tradeoff is measured.

Utility Objective

Let f_X be a model trained on raw features and f_Y be the same model class trained on shadow features. For metric M , such as AUC or F1, define:

$$\text{Retention}_M(C_K) = \frac{M(f_Y; Y, y)}{M(f_X; X, y) + \varepsilon_0},$$

where $\varepsilon_0 > 0$ prevents division by zero. A DSP candidate is useful only if it keeps retention above the target for the selected workload. For the current local benchmark, the key target is shadow AUC retention.

Empirical Leakage Metrics

DSP measures adversarial leakage through attack families, not vague claims. Let $\mathcal{A}_{rec}$ be a reconstruction attacker that receives shadow records and any allowed auxiliary raw/shadow pairs. Its reconstruction risk is:

$$R_{rec}(C_K) = \max_{A \in \mathcal{A}_{rec}} R^2(X, A(Y)).$$

Let $\mathcal{A}_{mem}$ be a membership attacker observing confidence scores. Membership advantage is:

$$Adv_{mem} = 2 \left| AUC_{mem} - \frac{1}{2} \right|.$$

Let $z = g(X)$ be a sensitive binary property. Property leakage is:

$$L_{prop}(C_K) = \max \left(AUC(A_{prop}(Y), z), 1 - AUC(A_{prop}(Y), z) \right).$$

The effective AUC form prevents an inverted but learnable sensitive property from being misreported as safe. A candidate is empirically acceptable only when utility remains high and R_{rec} , Adv_{mem} , and L_{prop} remain below the declared gate for the release object.

Random Projection Utility Intuition

For non-DP projection modes, the core utility intuition comes from random projection theory. If A_K is a suitable sub-Gaussian or orthogonal projection and $m = O(\epsilon^{-2} \log n)$, pairwise distances among n points are preserved up to multiplicative distortion $1 \pm \epsilon$ with high probability. Thus a shadow representation can retain enough geometry for downstream classification, retrieval, clustering, and ranking tasks.

Lemma 1 (Shadow geometry intuition). *For a finite dataset $X = \{x_1, \dots, x_n\}$, a properly scaled random projection $A_K \in \mathbb{R}^{d \times m}$ with sufficiently large m satisfies:*

$$(1 - \epsilon) \|x_i - x_j\|_2^2 \leq \|x_i A_K - x_j A_K\|_2^2 \leq (1 + \epsilon) \|x_i - x_j\|_2^2$$

for all pairs i, j with high probability.

Proof. This is the Johnson-Lindenstrauss preservation argument applied to the finite set of observed records. OSCIRIS does not rely on this lemma as a privacy proof. It explains why a lower-dimensional shadow can preserve task utility while reducing direct exposure of raw coordinates. $\square$

Formal DP Shadow Release

The strict DP shadow mechanism is:

$$M(D) = \text{clip}_B(X)A + \Xi,$$

where rows are clipped as:

$$\text{clip}_B(x) = x \cdot \min \left(1, \frac{B}{\|x\|_2 + 10^{-12}} \right),$$

A is a fixed projection matrix, and $\Xi_{ij} \sim \mathcal{N}(0, \sigma^2)$ is fresh independent Gaussian noise. Under replace-one neighboring datasets, the L_2 sensitivity is bounded by:

$$\Delta_2 \leq 2B \|A\|_{\text{op}}.$$

Theorem 1 (Scoped Gaussian DP shadow release). *Assume rows are clipped to norm at most B , the projection matrix A is fixed before release, noise is fresh and independent, and neighboring datasets differ by replacing one record. For any $0 < \delta < 1$, the release $M(D) = \text{clip}_B(X)A + \Xi$ satisfies the classical sufficient (ϵ, δ) -DP Gaussian bound:*

$$\epsilon \leq \frac{\Delta_2 \sqrt{2 \log(1.25/\delta)}}{\sigma}, \quad \Delta_2 \leq 2B \|A\|_{\text{op}}.$$

Proof. Clipping bounds each row norm by B . Replacing one record changes one clipped row by at most $2B$ in L_2 . Applying a fixed linear map A increases the norm by at most $\|A\|_{\text{op}}$, so the release sensitivity is at most

$2B \| A \|_{\text{op}}$. Adding fresh Gaussian noise with standard deviation σ gives the classical Gaussian mechanism sufficient bound. Any deterministic post-processing, including tanh or tensor compression, preserves the same DP bound. $\square$

Proposition 2 (Basic composition). *If the strict DP shadow mechanism is released T times and each release uses fresh independent noise with per-release parameters (ϵ_t, δ_t) , then the sequence of releases satisfies:*

$$\epsilon_{\text{total}} = \sum_{t=1}^T \epsilon_t, \quad \delta_{\text{total}} = \sum_{t=1}^T \delta_t.$$

Proof. This is the standard basic composition theorem for differential privacy. The current local accountant uses this conservative rule rather than a tighter advanced accountant. That choice is less aggressive but easier to audit. $\square$

Formal DP Model Release

The current strongest local formal result is a model-release path rather than a row-shadow release. Let $g_i(\theta)$ be a per-example gradient at epoch t . The mechanism clips gradients:

$$\bar{g}_i(\theta) = g_i(\theta) \cdot \min\left(1, \frac{B}{\|g_i(\theta)\|_2 + 10^{-12}}\right),$$

averages them:

$$\bar{g}(\theta) = \frac{1}{n} \sum_{i=1}^n \bar{g}_i(\theta),$$

and releases internally a noisy gradient:

$$\tilde{g}_t = \bar{g}(\theta_t) + \mathcal{N}(0, \sigma_t^2 I).$$

The model update is:

$$\theta_{t+1} = \theta_t - \eta \tilde{g}_t.$$

Only the final model is the selected release object. Because the final model is post-processing of the sequence of noisy gradients, its privacy budget is the composed budget of those gradient releases.

Theorem 2 (Scoped DP model release). *Assume per-example gradients are clipped to norm B , the training uses fresh independent Gaussian noise at each epoch, the neighboring relation is replace-one record, and basic composition is used across T epochs. The final model release is (ϵ, δ) -DP with:*

$$\epsilon = \sum_{t=1}^T \frac{\Delta_t \sqrt{2 \log(1.25/\delta_t)}}{\sigma_t}, \quad \delta = \sum_{t=1}^T \delta_t, \quad \Delta_t \leq \frac{2B}{n_{\text{train}}}.$$

Proof. At each epoch, replacing one record changes the average clipped gradient by at most $2B/n_{\text{train}}$, where n_{train} is the number of training records used in the DP optimizer. The Gaussian mechanism gives the stated per-epoch (ϵ_t, δ_t) bound. Basic composition sums those parameters over epochs. The final weights are deterministic post-processing of the noisy gradient sequence, so they inherit the same composed DP guarantee. $\square$

Result-Backed Math Check

The local result artifact `reports/dp_model_guarded_confirmed_stability/dp_model_sweep.json` records a stable scoped model-release candidate:

$$\epsilon = 25, \quad \delta = 10^{-5}, \quad B = 1.0, \quad T = 80, \quad \text{release object} = \text{model}.$$

The accountant records:

$$\Delta = \frac{2B}{n_{\text{train}}} = 0.0083333333.$$

Thus the implied training split size is:

$$n_{\text{train}} = \frac{2B}{\Delta} = \frac{2}{0.0083333333} = 240.$$

With basic composition:

$$\epsilon_t = \frac{25}{80} = 0.3125, \quad \delta_t = \frac{10^{-5}}{80} = 1.25 \times 10^{-7}.$$

Solving the Gaussian sufficient bound for σ :

$$\sigma = \frac{\Delta \sqrt{2 \log(1.25/\delta_t)}}{\epsilon_t} = \frac{0.0083333333 \sqrt{2 \log(1.25/(1.25 \times 10^{-7}))}}{0.3125} = 0.1514051314.$$

This matches the reported `noise_std=0.1514051314014696`. Therefore the theorem, accountant, and stored result agree for the scoped DP model-release claim.

Check	Reported Value	Interpretation
Formal DP claim	true	The local run satisfies the configured fresh-noise, clipping, and composition assumptions for the model release.
Epsilon upper bound	25.0	Matches requested total epsilon under basic composition.
Delta	10^{-5}	Split across 80 epochs as 1.25×10^{-7} per epoch.
Sensitivity	0.0083333333	Equals $2B/n_{\text{train}}$ with $B = 1.0$ and $n_{\text{train}} = 240$.
Noise standard deviation	0.1514051314	Matches the Gaussian calibration formula above.
AUC retention mean	0.9053981062	Passes the local utility gate ≥ 0.85 .
Property effective AUC mean	0.5603443366	Passes the local property leakage gate ≤ 0.60 .
Membership advantage mean	0.0698263889	Passes the local membership leakage gate ≤ 0.20 .
Reconstruction R2 mean	0.0	Not a row-release reconstruction proof; no row-shadow representation is selected for release.
Selected-release pass rate	$12/15 = 0.8$	Meets the local stability target.

The production proof suite separately records a five-seed local proof track: formal DP claim 5/5, epsilon 25, delta 10^{-5} , utility pass rate 0.8, and mean AUC retention 0.913546. This supports the same scoped model-release claim, while still requiring independent review before enterprise-grade privacy claims.

DSP Claim Boundary

Under the documented fresh-noise, clipping, neighboring-dataset, and composition assumptions, the scoped DP model-release mechanism satisfies the reported (ϵ, δ) bound. Local evidence shows useful task retention on the enterprise synthetic benchmark. DSP provides a measurable privacy layer with formal DP modes, empirical leakage gates, and a path to enterprise-grade validation through independent review and distributed-system testing.

Verification and Trust Model

Verification is the central trust challenge in decentralized AI compute. OSCIRIS does not attempt to prove every floating-point operation on-chain. Instead, it uses risk-tiered verification:

- low-risk public jobs use reputation, spot checks, and output hashes;
- medium-risk jobs use redundancy, hidden canary tasks, and verifier review;
- high-risk enterprise jobs use vetted providers, signed runtime images, TEE attestation where available, checkpoint hashes, signed logs, evaluator nodes, and challenge windows.

Smart contracts verify objective coordination facts:

- provider registration and stake sufficiency,
- job terms and manifest hashes,
- escrow funding,
- attestation and runtime measurement hashes,
- checkpoint and output artifact hashes,
- verifier signatures and quorum status,
- challenge-window expiry,
- slashing and insurance-pool routing.

Off-chain verifiers perform the expensive work: attestation validation, benchmark challenges, evaluation-set checks, checkpoint review, artifact integrity, and privacy or policy checks. On-chain contracts record the results and enforce payment or penalty.

Verifier Acceptance Rule

For job J , verifier V_k emits:

$$v_k(J) = (h_{\text{eval}}, \text{score}_k, \text{pass}_k, \text{Sig}_{V_k}).$$

Let w_k be verifier weight, derived from stake, reputation, specialization, or allowlisted enterprise status. The job passes verifier quorum if:

$$\sum_{k=1}^{\ell} w_k \cdot \text{pass}_k \geq q_J \quad \text{and} \quad \text{validsig}(\Sigma_V) = 1.$$

The acceptance predicate is:

$$\text{valid}(\rho_J) = \mathbf{1}\{h_{\text{out}} = \text{Hash}(A_{\text{out}}), h_{\text{eval}} = \text{Hash}(E_{\text{eval}}), \sum_k w_k \text{pass}_k \geq q_J, t < t_{\text{challenge_end}}\}.$$

This design lets OSCIRIS verify objective commitments on-chain while keeping task-specific evaluation off-chain.

Redundancy and Spot-Check Detection

For workloads that permit redundancy, assign the same hidden canary or checkpoint validation to k independent providers or verifiers. If each independent check detects a bad output with probability p_d , then the probability that all checks miss the bad output is:

$$\mathbb{P}[\text{miss}] \leq (1 - p_d)^k.$$

Thus redundancy gives an exponential reduction in missed invalid work when the independence assumption is reasonable. OSCIRIS uses higher k for privacy-sensitive or high-value jobs and lower k for low-risk public jobs.

False Acceptance Under Verifier Error

If each verifier independently accepts an invalid output with probability p_f , and a job requires at least q accepting votes out of ℓ equal-weight verifiers, then:

$$\mathbb{P}[\text{false accept}] = \sum_{r=q}^{\ell} \binom{\ell}{r} p_f^r (1 - p_f)^{\ell-r}.$$

This is not a universal security theorem because verifier errors may be correlated. It is the baseline sizing equation for quorum design. Enterprise pools reduce correlation through independent verifier operators, diverse evaluation methods, and separate runtime attestation paths.

Challenge Game

Let b_J be the job fee, c_h be the honest provider cost, c_m be the malicious or shortcut cost, p_d be the probability of detection during verification and challenge, and L_J be the slash amount. A rational provider prefers honest execution if:

$$b_J - c_h \geq (1 - p_d)(b_J - c_m) - p_d L_J.$$

Equivalently, the slash amount satisfies:

$$L_J \geq \frac{c_h - c_m}{p_d}.$$

This equation is central to the OSCIRIS economics. If detection probability is low, stake penalties increase or the job routes to a vetted provider pool. For highly sensitive workloads, OSCIRIS increases p_d through vetted providers, TEEs, canary tasks, redundancy, signed logs, and independent verifiers.

Slashing and Insurance Routing

Let $s_i(t)$ be provider P_i 's bonded ZEN stake at time t . For violation type v , define:

$$L_{J,i}(v) = \min(s_i(t), \lambda_v b_J \omega_{\pi_J}),$$

where λ_v is the violation multiplier and ω_{π_J} is a privacy-tier multiplier. A public low-risk timeout carries a smaller penalty than a proven privacy violation. The insurance pool evolves as:

$$I_{t+1} = I_t + \sum_i L_{J,i} - \text{compensation}_J - \text{verifier_rewards}_J - \text{audit_budget}_J.$$

The protocol maintains $I_t \geq I_{\min}$ for active enterprise tiers. If the pool falls below the reserve threshold, new high-risk jobs pause or require additional provider collateral.

Blockchain Architecture and Horizen Implementation

Horizen is a practical implementation target because current Horizen documentation describes it as an EVM-native blockchain and privacy app ecosystem that rolls up to Base . Horizen architecture documentation describes Horizen as an EVM L3 powered by OP Stack on top of Base . Horizen also documents Solidity smart contract deployment through Foundry and Hardhat .

The initial OSCIRIS contract suite includes:

- **ProviderRegistry**: provider identity, endpoint, hardware class, jurisdiction labels, supported privacy tiers, public keys, and status;
- **ZenStakeVault**: ZEN staking, tier eligibility, lockups, withdrawal rules, and reward-weighted stake caps;
- **JobEscrow**: USDC job funding, job terms hash, SLA terms, provider assignment, and payout conditions;
- **AttestationRegistry**: approved runtime hashes, attestation report hashes, expiry, and verifier signatures;
- **VerificationOracle**: verifier submissions, score hashes, checkpoint hashes, output hashes, pass/fail results, and quorum status;
- **ChallengeManager**: dispute windows, evidence hashes, challenger bonds, and resolution status;
- **SlashingManager**: objective violation types, slash amounts, and insurance-pool routing;
- **InsurancePool**: slashed ZEN reserves for customer compensation, verifier rewards, audits, and ecosystem support.

Contract Invariants

Contract	Core State	Invariant
ProviderRegistry	provider identity, keys, tier, status	Only active providers with valid keys and tier metadata can accept routed work.
ZenStakeVault	deposits, lockups, effective stake, pending slash	Withdrawable stake equals deposited stake minus locked and slash-pending stake.
JobEscrow	funded jobs, budgets, deadlines, assigned providers	Stablecoin funds cannot be paid until receipt and quorum predicates pass.
AttestationRegistry	runtime hashes, expiry, verifier signatures	Expired or unapproved runtime measurements cannot satisfy high-tier jobs.
VerificationOracle	verifier reports, score hashes, quorum state	One verifier cannot unilaterally finalize a job above the configured threshold.
ChallengeManager	challenge bonds, evidence hashes, resolution	Final settlement is blocked while a valid challenge is inside its window.
SlashingManager	violation classes, slash multipliers, routing	Slash amounts are capped by bonded stake and routed by explicit policy.
InsurancePool	reserves, compensation, verifier rewards, audits	Enterprise-tier job admission must respect reserve thresholds.

Proposition 3 (Escrow safety). *Assume the contract implementation correctly enforces the receipt predicate, quorum predicate, and challenge-window predicate. Then JobEscrow cannot release customer funds for job J unless J has a valid receipt, sufficient verifier approval, and no successful unresolved challenge.*

Proof. The payout transition is guarded by the conjunction $\text{valid}(\rho_J) = 1$, $\sum_k w_k \text{pass}_k \geq q_J$, and $\text{challenge}(J) = 0$. If any predicate is false, the transition is not enabled. Therefore funds remain escrowed, become refundable on expiry, or move through the slashing/challenge resolution path. $\square$

Horizen token documentation lists ZEN on Base and Horizen, and also lists USDC contracts on Base and Horizen . OSCIRIS uses ZEN as collateral and USDC as the stable settlement asset. The exact deployment follows the then-current official contract addresses and bridge support at implementation time.

Token Economics and Incentive Design

OSCIRIS uses blockchain for economic enforcement, not for raw AI execution. The initial economic model is:

- providers stake ZEN to join the network and access job tiers;
- customers fund jobs in USDC or stable-value settlement rails;
- providers earn stablecoin rewards for verified work;
- failed attestation, invalid proofs, SLA failure, malicious updates, or proven privacy violations slash provider ZEN;
- slashed ZEN is routed to an insurance and ecosystem pool rather than burned by default;
- reputation is non-transferable and derived from verified performance.

This model avoids using a new speculative native token before the network proves real compute demand. A future OSCIRIS-native token may be considered only if the network needs independent governance, cross-chain incentive expansion, or community ownership beyond the initial Horizen-aligned design.

Stake Tiers

Provider stake is reward-weighted rather than unlimited. A provider can hold more ZEN, but only a capped amount counts toward job priority. This prevents capital concentration from overwhelming compute quality.

Tier	Provider Type	Economic Role
Tier 1	Public or low-risk node	Low stake, limited public workloads, spot-check verification.
Tier 2	Prosumer GPU node	Medium stake, batch inference, embeddings, synthetic-data jobs, stronger uptime requirements.
Tier 3	Vetted workstation or cluster	Higher stake, private fine-tuning, checkpointing, audit logs, higher verifier scrutiny.
Tier 4	Confidential or enterprise node	Highest stake, TEE attestation, regulated workloads, allowlisted participation, strict dispute rules.

Reward Formula

A simple first reward model is:

$$\text{reward} = \text{job fee} \times \text{work share} \times \text{quality score} \times \text{uptime score} \times \text{reputation multiplier}$$

The job fee is paid in stable-value rails. Work share is measured by accepted compute contribution. Quality score is derived from task-specific verification. Uptime score penalizes unreliable providers. Reputation grows slowly and cannot be bought instantly with stake.

Reward Allocation

For job J , let F_J be the stablecoin fee available for providers after protocol fees, verifier fees, and insurance reserve contributions. Provider P_i 's normalized work share is:

$$\omega_i = \frac{\widehat{g}_i}{\sum_{k \in A_J} \widehat{g}_k},$$

where $\widehat{g}_i$ is accepted compute contribution and A_J is the set of accepted providers. Define quality $Q_i \in [0, 1]$, uptime $U_i \in [0, 1]$, and reputation multiplier:

$$M_i = 1 + \min(\rho_{\max}, \rho r_i).$$

The provider reward is:

$$R_{J,i} = F_J \cdot \frac{\omega_i Q_i U_i M_i}{\sum_{k \in A_J} \omega_k Q_k U_k M_k}.$$

This makes rewards proportional to useful verified work, not merely raw stake. Stake controls eligibility and penalty capacity; it does not dominate payout.

Effective Stake Cap

Let s_i be raw ZEN stake and $s_{\max}(\pi)$ be the reward-weighted cap for privacy tier π . Effective stake is:

$$s_i^{eff}(\pi) = \min(s_i, s_{\max}(\pi)).$$

Assignment priority can depend on s_i^{eff} , but not on unlimited stake. This reduces plutocratic routing and keeps quality, uptime, privacy history, and verifier outcomes central to provider ranking.

Sustainable Pricing Condition

For each job, the customer fee must cover compute, verification, retries, storage, insurance reserve, and protocol margin:

$$b_J \geq C_{\text{compute}} + C_{\text{verify}} + C_{\text{retry}} + C_{\text{storage}} + C_{\text{insurance}} + M_{\text{protocol}}.$$

If this inequality does not hold, the job is economically unsafe even if it is technically executable. OSCIRIS prices jobs by quality-adjusted cost, not by the cheapest GPU-hour headline number.

Provider Participation Condition

Provider P_i participates rationally when expected reward exceeds expected cost plus expected penalty risk:

$$\mathbb{E}[\Pi_i] = \mathbb{E}[R_i] - C_i - \mathbb{P}[\text{fault}_i] \mathbb{E}[L_i] > 0.$$

Good providers receive positive expected profit. Bad or unreliable providers receive negative expected profit once detection and slashing are included. This is the economic reason for staking ZEN rather than using it as the customer payment asset.

Why No Native Token Initially

A new native token is not required to validate the first economic loop. The minimum viable loop is:

USDC demand → verified jobs → provider rewards → ZEN collateral → slashable accountability.

An OSCIRIS-native token becomes justified only if there is a real governance or coordination function that ZEN and stablecoins cannot serve: cross-ecosystem incentive expansion, protocol ownership, verifier-market governance, or multi-chain security. Launching a native token before usage would weaken the economic story by making speculation the product.

Governance, Compliance, and Enterprise Controls

OSCIRIS uses governance and compliance controls to make the protocol usable for regulated buyers. Enterprise trust is not delegated to token incentives alone. The control layer defines who can execute sensitive work, how data policies are attached to jobs, how incidents are handled, and which audit evidence is exported after execution.

Control Area	Protocol Treatment
Provider onboarding	Vetted pools, jurisdiction labels, supported privacy tiers, key registration, runtime requirements, and hardware metadata.
Data governance	Dataset manifests, consent metadata, retention policy, allowed jurisdictions, privacy tier, and release-object scope.
Audit exports	Job terms, manifest hashes, runtime measurements, verifier receipts, challenge status, settlement records, and privacy reports.
Incident response	Challenge windows, evidence hashes, slashing routes, customer compensation rules, and provider suspension paths.
Enterprise pools	Allowlisted providers, stricter verifier quorum, confidential-runtime requirements, insurance thresholds, and legal terms.
Upgrade policy	Conservative contract upgrades, explicit parameter changes, crypto-agility metadata, and independent review for sensitive controls.

The enterprise mode is therefore semi-permissioned even if the broader protocol can support more open provider participation. This keeps early adoption aligned with procurement, compliance, and operational reality.

Validation, Proof Status, and Claim Ledger

OSCIRIS currently has a local validation harness, not production proof. The local lab validates DSP behavior, scoped DP model release, manifest integrity, adversarial update logic, and simulated reliability. It does not yet prove CUDA/NVIDIA throughput, multi-node network reliability, decentralized GPU economics, or large-model advantage.

Track	Current Status	Interpretation
Formal privacy guarantees	Local partial pass	Scoped DP model-release claim exists under documented assumptions. Independent review required.
Enterprise-grade privacy	Local evidence	Needs governance controls, red-team evidence, key management, retention, and audit workflows.
Adversarial robustness	Local simulation	Robust aggregation and malicious worker logic are simulated locally. Real worker tests are still required.
Real node reliability	Local simulation	Churn and retry behavior are simulated. Real 3-5 node tests are still required.
CUDA/NVIDIA performance	Blocked externally	Requires real NVIDIA GPU benchmarks with CUDA, PyTorch, vLLM or TensorRT-LLM where relevant.
Decentralized economics	Blocked externally	Requires measured provider pricing, throughput, verification overhead, retry cost, and quality metrics.
Large-model advantage	Blocked externally	Requires 7B-class QLoRA or inference benchmarks on real GPU hardware.

Claim Ledger

The following ledger prevents the whitepaper from overclaiming. Each claim is classified as formal, protocol-level, local empirical, or roadmap.

Claim	Status	Current Basis	Remaining Gap
DSP can preserve task utility in shadow space	Local empirical	Local benchmark shows useful AUC retention for selected candidates.	More datasets, external review, production workloads.
DSP row-shadow releases can have scoped DP bounds	Formal under assumptions	Gaussian mechanism with clipping, fresh noise, sensitivity accounting, and basic composition.	Stable utility/privacy gate for row-shadow release.
DSP model release has the strongest current formal result	Formal under assumptions plus local evidence	DP full-batch gradient model release at epsilon 25, delta 10^{-5} , 15-seed local sweep.	Independent proof review and distributed deployment validation. Add PQC or hybrid PQC key establishment, encryption, and signatures around DSP artifacts and protocol messages.
DSP is post-quantum secure	Not claimed	DSP is a representation and privacy layer, not a PQC primitive.	
OSCIRIS does not need blockchain for raw AI execution	Architectural	AI compute is off-chain; chain stores commitments, escrow, signatures, and challenge state.	Contract implementation and audit.
Blockchain is necessary for open-network economics	Protocol-level	Stake, escrow, slashing, verifier rewards, insurance, and portable reputation require shared settlement.	Testnet deployment and adversarial trials.
ZEN can function as provider collateral	Economic design	ZEN stake gates access and absorbs penalties while USDC handles payments.	Live liquidity, bridge, custody, and legal review.
Stablecoin rewards improve enterprise procurement	Product/economic design	Customers avoid volatile payment exposure; providers receive predictable settlement.	Payment integration and customer validation.
Verifier quorum reduces invalid-output acceptance	Mathematical model	Binomial false-acceptance sizing and challenge-game economics.	Correlated verifier failures and real verifier-market testing.
Redundancy lowers missed bad-output probability	Mathematical model	$(1 - p_d)^k$ miss-probability bound under independence.	Independence validation under real nodes.
Local node reliability simulation supports control logic	Local simulation	Production proof suite simulates retry/churn behavior.	Real multi-node deployment across network boundaries.
Decentralized GPU economics are sustainable	Roadmap	Scenario model exists for cost-quality comparison.	Measured GPU baselines, retries, bandwidth, verification

Claim	Status	Current Basis	Remaining Gap
OSCIRIS can support enterprise/sovereign workloads	Roadmap with local evidence	Privacy tiers, audit design, DSP, verifier model, and managed workspace GTM.	overhead, and provider pricing. Security review, legal workflows, pilots, and production operational controls.

Roadmap

Timeline	Milestone	Deliverables
0-3 months	Prototype demo	Provider registration, ZEN staking simulation, USDC escrow flow, verification records, challenge windows, and DSP reports.
3-6 months	Testnet contract suite	Solidity contracts for registry, staking, escrow, attestation records, verifier submissions, slashing, and insurance routing.
6-9 months	Vetted provider pilot	3-5 real distributed nodes, signed job receipts, checkpoint hashes, provider reliability reports, and pilot AI tasks.
9-12 months	Enterprise pilot	Private RAG, embeddings, evaluation, and fine-tuning pilot with audit logs, privacy tiers, and customer feedback.
12-18 months	Confidential and federated layer	TEE integration, secure aggregation, verifier marketplace, and privacy-governed provider pools.
18+ months	Open protocol expansion	Permissionless provider onboarding, cross-border settlement, advanced verification, and expanded ecosystem governance if usage justifies it.

Risks and Limitations

OSCIRIS is ambitious and must be validated incrementally. The protocol does not replace hyperscale clusters for frontier-model pretraining in its initial form. Encrypted storage alone does not guarantee compute privacy. DSP shadow-space benchmarks are evidence, not universal privacy guarantees. Formal DP claims are scoped to specific mechanisms and release objects. Real provider reliability, GPU economics, throughput, network conditions, and enterprise procurement constraints still require external pilots and independent review.

Production readiness requires:

- real NVIDIA GPU benchmarks,
- real multi-node reliability measurements,
- external security and privacy review,
- measured decentralized cost-to-quality comparisons,
- pilot customer feedback,
- clear legal, data-processing, and governance controls.

Conclusion

OSCIRIS Protocol is an attempt to make decentralized AI compute useful for serious enterprise and sovereign workloads. Its core thesis is that the future compute market needs more than raw GPU aggregation. It needs privacy-aware scheduling, verifiable execution, economic accountability, auditable workflows, and stable-value settlement.

The protocol begins as a managed, semi-permissioned secure compute network for AI inference, RAG, evaluation, and fine-tuning. It can then expand into confidential compute, federated learning, sovereign AI zones, and broader decentralized provider participation as verification, privacy, reliability, and economics are proven.

22

Johnson, W. B. and Lindenstrauss, J. Extensions of Lipschitz mappings into a Hilbert space. Contemporary Mathematics, 1984.

Dwork, C. and Roth, A. The Algorithmic Foundations of Differential Privacy. Foundations and Trends in Theoretical Computer Science, 2014.

National Institute of Standards and Technology. Announcing Approval of Three Federal Information Processing Standards for Post-Quantum Cryptography. <https://csrc.nist.gov/News/2024/postquantum-cryptography-fips-approved>

National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard. <https://csrc.nist.gov/pubs/fips/203/final>

National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard. <https://csrc.nist.gov/pubs/fips/204/final>

National Institute of Standards and Technology. FIPS 205: Stateless Hash-Based Digital Signature Standard. <https://csrc.nist.gov/pubs/fips/205/final>

Akash Network. AkashML: Managed AI Inference on the Decentralized Supercloud. <https://akash.network/blog/akashml-managed-ai-inference-on-the-decentralized-supercloud/>

io.net. IO Cloud documentation. <https://io.net/docs/guides/clouds/deploy-cluster>

Aethir. Enterprise GPU-as-a-Service. <https://enterprise.aethir.com/>

Gensyn. Official Docs: Decentralized Protocol for Machine Learning. <https://docs.gensyn.ai/>

Bittensor. Understanding Subnets and Validating in Bittensor. <https://docs.learnbittensor.org/subnets/understanding-subnets>

Tong, C. et al. Parallax: Efficient LLM Inference Service over Decentralized Environment. arXiv, 2025. <https://arxiv.org/abs/2509.26182>

Chrapek, M., Copik, M., Mettaz, E., and Hoefler, T. Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs. arXiv, 2025. <https://arxiv.org/abs/2509.18886>

NVIDIA Developer Blog. Confidential Computing on NVIDIA H100 GPUs for Secure and Trustworthy AI. <https://developer.nvidia.com/blog/confidential-computing-on-h100-gpus-for-secure-and-trustworthy-ai/>

IPFS Docs. Privacy and Encryption. <https://docs.ipfs.tech/concepts/privacy-and-encryption/>

Filecoin Docs. Proofs: Proof of Replication and Proof of Spacetime. <https://docs.filecoin.io/basics/the-blockchain/proofs>

Horizen Documentation. Horizen Overview. <https://docs.horizen.io/>

Horizen Documentation. Architecture. <https://docs.horizen.io/overview/architecture/>

Horizen Documentation. Deploy a Contract. <https://docs.horizen.io/overview/deploy-contract/>

Horizen Documentation. Tokens. <https://docs.horizen.io/overview/token/>

Mazmudar, M., Veitch, S., and Akhavan Mahdavi, R. Peer2PIR: Private Queries for IPFS. arXiv, 2024. <https://arxiv.org/abs/2405.17307>

Sridhar, S. et al. Content Censorship in the InterPlanetary File System. arXiv, 2023. <https://arxiv.org/abs/2307.12212>